Department of Nano Science Technology
Faculty of Technology
Wayamba University of Sri Lanka

Centralized Workshops and Laboratory Allocation Platform

Submitted to: Dr. U.S. Liyanarachchi
Module: NANO2142

Submitted by: Group 1

Date of Submission

05 Nov 2026

Contents

Abstract 3
1. Introduction and Background 3
1.1 Project scope 4
1.2 Existing and proposed process 4
2. Problem Definition and Objectives 4
2.1 Problem definition 4
2.2 Project aim 4
2.3 Objectives 5
2.4 Success criteria 5
3. System Requirements 5
3.1 Actors and responsibilities 5
3.2 Functional requirements 6
3.3 Non-functional requirements 6
4. System Design 7
4.1 Main use cases 7
4.2 High-level architecture 7
4.3 UI/UX design and student workflow 8
4.4 Edit and administrator workflows 10
4.5 ER diagram and database schema 10
4.6 Class diagram and OOP design 12
5. Implementation Details and Key Decisions 13
Assumptions and Design Considerations 13
5.1 Allocation and slot rules 13
5.2 Database consistency 14
5.3 Email behavior 14
5.4 Settings, migration and deployment 14
6. Testing Strategy and Results 14
6.1 Key test cases 15
6.2 SQL query verification 15
7. Limitations and Possible Future Improvements 15
7.1 Deployment considerations 16
8. Individual Contributions 16
9. Conclusion 16
10. References 17

Abstract

SYNCLAB (Centralized Workshop and Laboratory Allocation Platform) is a C++ desktop application developed to organize laboratory and workshop allocation within the Faculty of Technology. It replaces a paper-based request workflow with a guided graphical interface for selecting a facility, date and available time slots, entering student and project information, and creating an allocation request. Each request receives an Allocation Key for later retrieval, update or deletion.

The application uses C++17, Qt 6, SQLite, qmake, Gemini 4 Argon and ChatGPT 6 luna. Its design separates the user interface, allocation rules, database access, student-record model and utility components. Four related tables store facility categories, student profiles, allocation records and individual reserved slots. Availability checks and transaction-protected database writes help maintain consistent records and prevent conflicting active allocations.

Facility administrators can review records, manage status and remarks, filter by date, edit or delete selected records, store HOD/Dean email addresses, prepare email drafts. Email actions open a draft in the default email application; SYNCLAB does not verify sending or delivery. Supplied project evidence reports a successful release build, 10 passing integration test groups with 69 assertions, 20 SQL checks and 19 UI update/delete assertions. The project remains a local prototype with lightweight access control.

1. Introduction and Background

Laboratories and workshops support practical classes, project work, research and other academic activities within the Faculty of Technology. Since these facilities are centralized, their use must be coordinated between students and the staff responsible for approving access.

In the manual process described for this project, students complete a physical request form and seek authorization from the Technical Officer or Laboratory In-Charge, the Head of the Department (HOD), and the Dean. Paperwork and communication between several people can make it difficult to check availability quickly, maintain consistent records and identify scheduling conflicts.

SYNCLAB was developed as a centralized desktop prototype to guide allocation requests and provide a structured place to review records. It is intended for a dedicated terminal near the relevant laboratory or workshop, similar in concept to an ATM. Because many students may centralize the terminal, the design includes an Allocation Key for a student request, an Administrator Passcode for a facility dashboard and a Master Passcode for system settings.

1.1 Project scope

- Create, retrieve, update and delete student allocations.
- Select a facility, date and available time slot(s), with calendar-based status visibility.
- Support facility administrator review, status and remark management, date filtering, email-draft preparation.
- Provide appearance settings and facility administrator passcode management.
- Store related data locally in SQLite and protect related writes with transactions.

1.2 Existing and proposed process

The existing process relies on a paper request and a sequence of staff decisions. SYNCLAB digitizes request entry, schedule checking and record management, while final authorization remains the responsibility of the required university authorities.

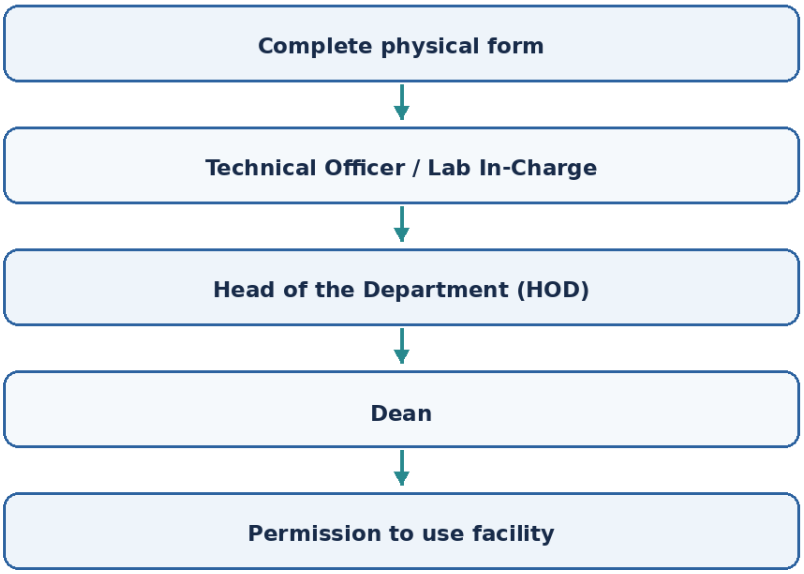


Figure 1 Existing manual approval chain

2. Problem Definition and Objectives

2.1 Problem definition

The system must store student and project details alongside the requested facility, date and time slot; prevent conflicting active allocations; and allow authorized staff to review requests and their status. The schedule must remain consistent when requests are created, changed, rejected or deleted. Access to individual student records and administrator functions must also be restricted because the system is designed for a centralized terminal.

2.2 Project aim

To develop a centralized, terminal-based desktop application that supports structured laboratory and workshop allocation, availability checking, request management and administrative review.

2.3 Objectives

- Provide guided screens for facility, date and available time-slot selection.
- Collect student, supervisor and project details and generate a unique Allocation Key.
- Allow students to retrieve their own allocation using the key and update or delete it.
- Store facility, student, allocation and slot data in a structured SQLite database.
- Check slot availability during create/update operations and protect writes with transactions.
- Enable facility administrators to review records, manage status/remarks, filter dates, edit/delete, prepare email drafts.
- Demonstrate object-oriented programming concepts, including encapsulation, inheritance, polymorphism and Qt event handling.

2.4 Success criteria

Area	Evidence of completion
Student workflow	Guided facility, date/time, details, confirmation and key-based editing.
Schedule integrity	Conflicting active facility/date/slot requests are rejected.
Administration	Facility-scoped dashboard supports record review, status/remarks, filtering, email drafts.
Data integrity	Related allocation and slot changes use transaction-protected database operations.
Verification	Build, integration, SQL and selected UI update/delete checks are recorded.

3. System Requirements

3.1 Actors and responsibilities

Actor	Responsibilities
Student	Create an allocation; enter details; retain the Allocation Key; retrieve, update or delete their own request.
Facility administrator	Review allocations for the selected facility; manage status and remarks; filter, edit/delete, prepare email drafts.
System administrator	Access system settings to change appearance options and facility administrator passcodes.
HOD and Dean	Act as external authorities who review requests and communicate approval or rejection.

3.2 Functional requirements

ID	Requirement	Implementation evidence
FR-01	Select a laboratory/workshop category.	Facility catalog and selection interface.
FR-02	Choose a date and free time slot(s).	Calendar, slot controls and availability checks.
FR-03	Save student, supervisor and project information.	Allocation form and StudentRecord model.
FR-04	Retrieve a request for editing/deletion using a key.	Allocation Key generation and key-based edit flow.
FR-05	Review status and save remarks.	Administrator table and status/remark controls.
FR-06	Filter allocations by facility and date.	Facility-scoped queries and date filtering.
FR-07	Maintain passcodes and appearance settings.	Settings interface and QSettings.
FR-08	Prepare pending/rejection email drafts and store HOD/Dean addresses.	Email settings and mailto-based draft preparation.

3.3 Non-functional requirements

Quality	Requirement
Usability	Guided, ordered workflow, clear actions and validation feedback.
Reliability	Transaction rollback and foreign-key integrity help avoid partial writes.
Security/privacy	Separate student, facility administrator and system administrator credentials; prototype-level access control.
Maintainability	Separate UI, allocation rules, database access, models and utilities.
Portability	Windows bundle includes required Qt modules and the QSQLITE driver; source builds require a compatible Qt 6 kit.
Data integrity	Active slot conflicts are checked on create/update; rejected allocations release slot occupancy.

4. System Design

4.1 Main use cases

ID / Use case	Actor	Summary
UC-01 Create allocation	Student	Select facility/date/slot(s), enter details, save a Pending request and receive a key.
UC-02 Edit/delete allocation	Student	Enter the key, view pre-filled information, then update or delete the request.
UC-03 Manage requests	Facility administrator	Open the dashboard, review records, set status/remarks, filter, edit/delete, prepare emails.
UC-04 Inspect allocations by date	Student / administrator	Use the calendar and date filter to inspect status and allocation information.
UC-05 Configure settings	System administrator	Use protected settings for appearance and administrator passcode management.

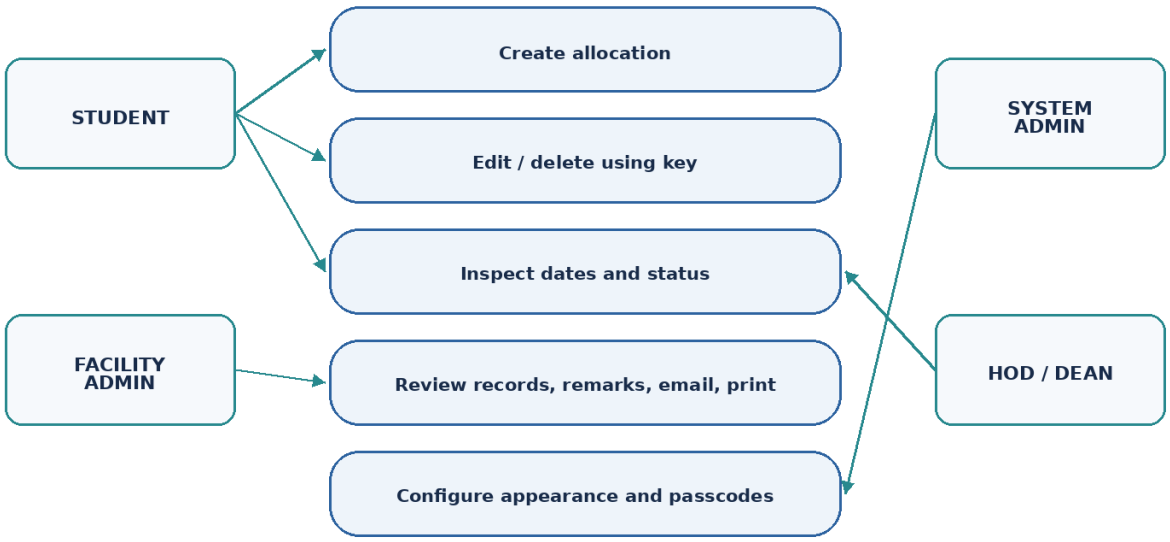


Figure 2 High-level use-case overview. Conceptual model

4.2 High-level architecture

SYNCLAB separates the graphical interface, application rules, persistence contract, SQLite implementation, domain model and utilities. This separation reduces the need to place all validation and database operations directly inside screen code.

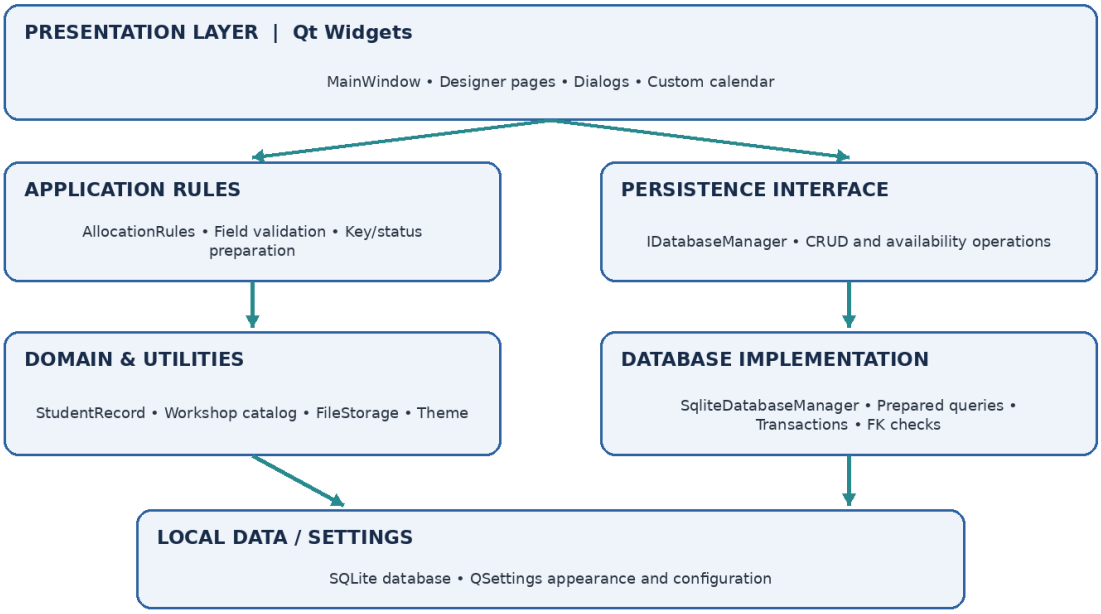


Figure 3 High-level architecture based on the supplied active-project documentation

4.3 UI/UX design and student workflow

The Welcome GUI offers New Allocation, Edit Allocation and Settings. A new request proceeds through facility selection, date/time selection and a details/confirmation screen. The date calendar uses A (Approved), P (Pending) and R (Rejected) indicators. Four fixed time slots are supported: 8:30–10:30, 10:30–12:30, 1:30–3:30 and 3:30–5:30. An active request reserves its selected slot for the same facility and date; rejected requests release their slots.

The allocation form requires name, index number, department, email address, supervisor name and project name. Description is optional. Confirmation saves the request as Pending and displays the Allocation Key. The key is required to retrieve the saved request for editing or deletion.

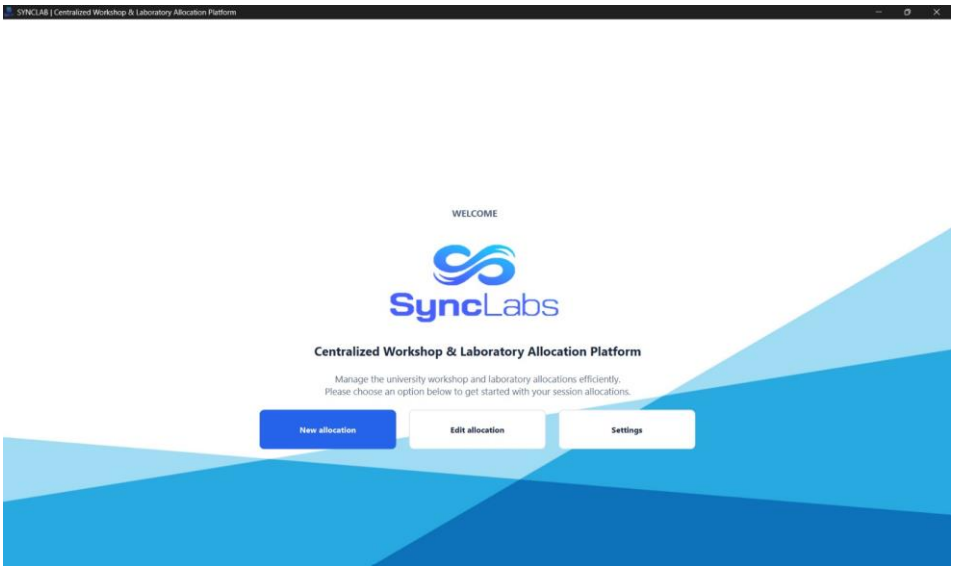


Figure 4 Welcome GUI. (New Allocation, Edit Allocation and Settings)

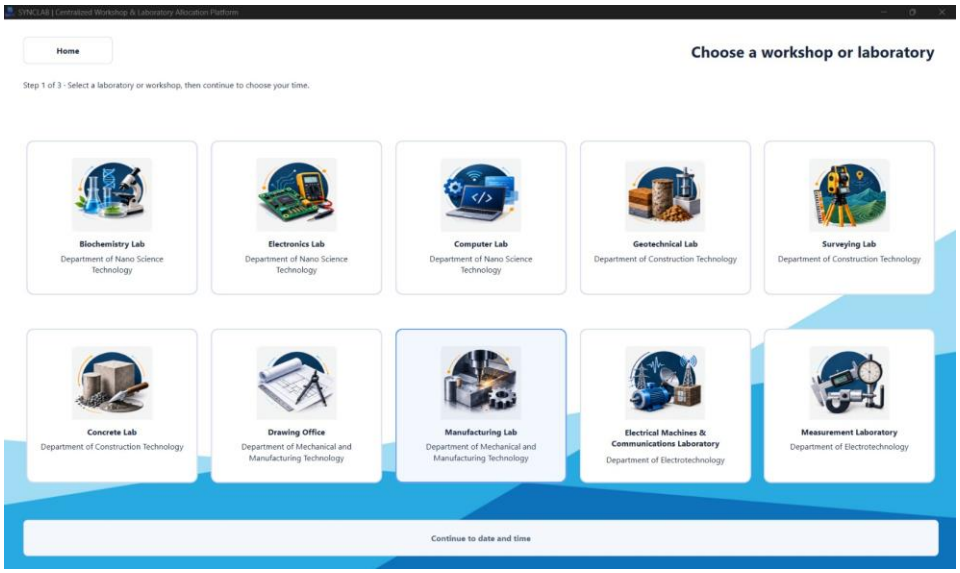


Figure 5 Laboratory and workshop list

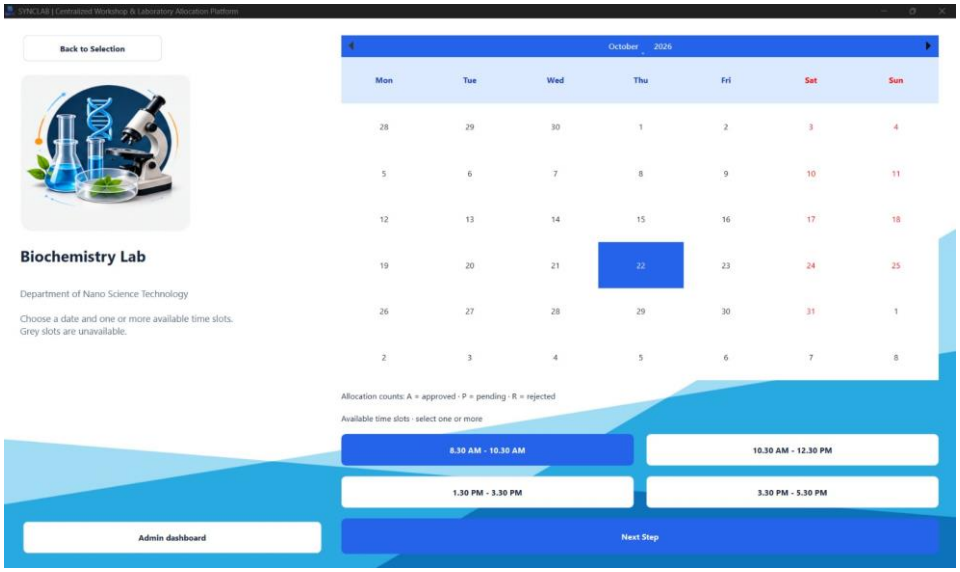


Figure 6 Date/time screen showing A/P/R indicators and the four available time periods

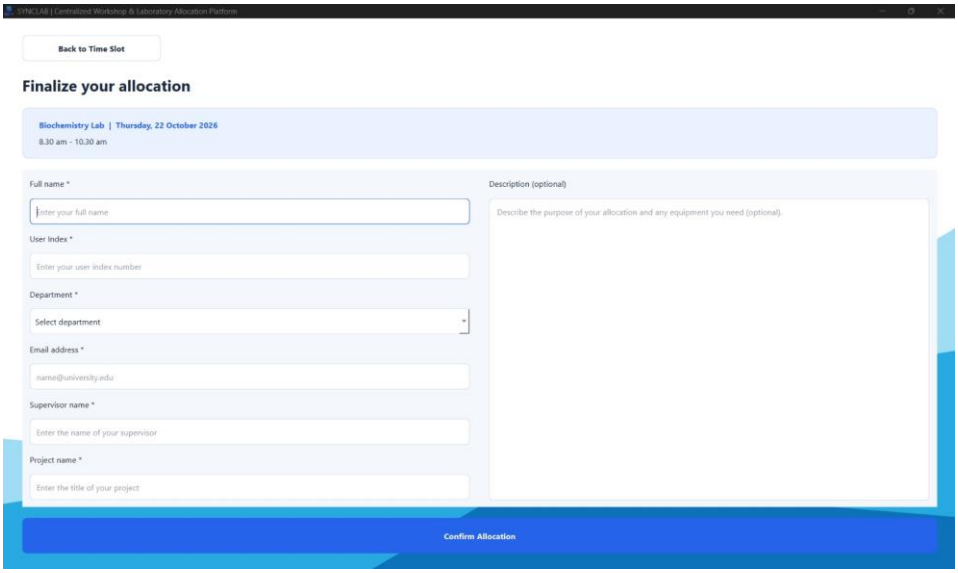


Figure 7 Allocation form and a separate confirmation popup. Use fictional demonstration data.

4.4 Edit and administrator workflows

To edit a request, a student selects Edit Allocation and enters the Allocation Key. A matching request is loaded with its facility, date/time and previously entered details pre-filled. Update Allocation saves changes and returns the request to Pending for review; Delete Allocation removes the request and its related slot rows.

The facility administrator opens the dashboard for the selected facility by entering its Administrator Passcode. The dashboard provides a calendar and an allocation table containing the key, creation date/time, student and index details, department, email, requested date/time, description, remarks and status. A row must be selected before the Edit Selected, Delete Selected, Pending Email Draft or Rejection Email Draft actions can be used.

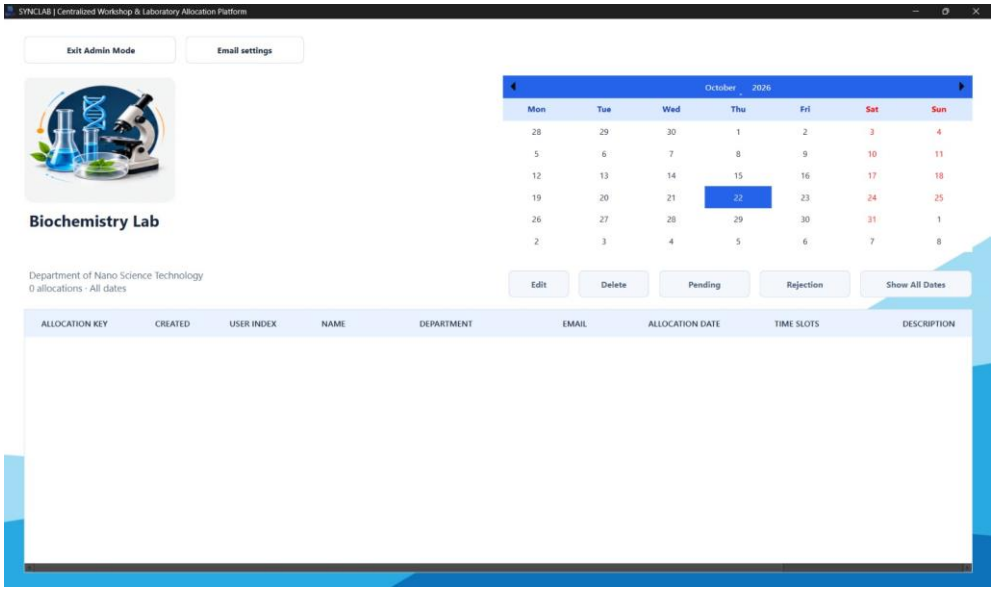


Figure 8 Dashboard showing the facility name, calendar, table, status/remark controls and action buttons.

4.5 ER diagram and database schema

SQLite stores four related tables: workshops, students, records and booking slots. Facility and student information are separated from allocation-specific values. Each selected time slot is stored as a child row, rather than as a repeating group inside the allocation record.

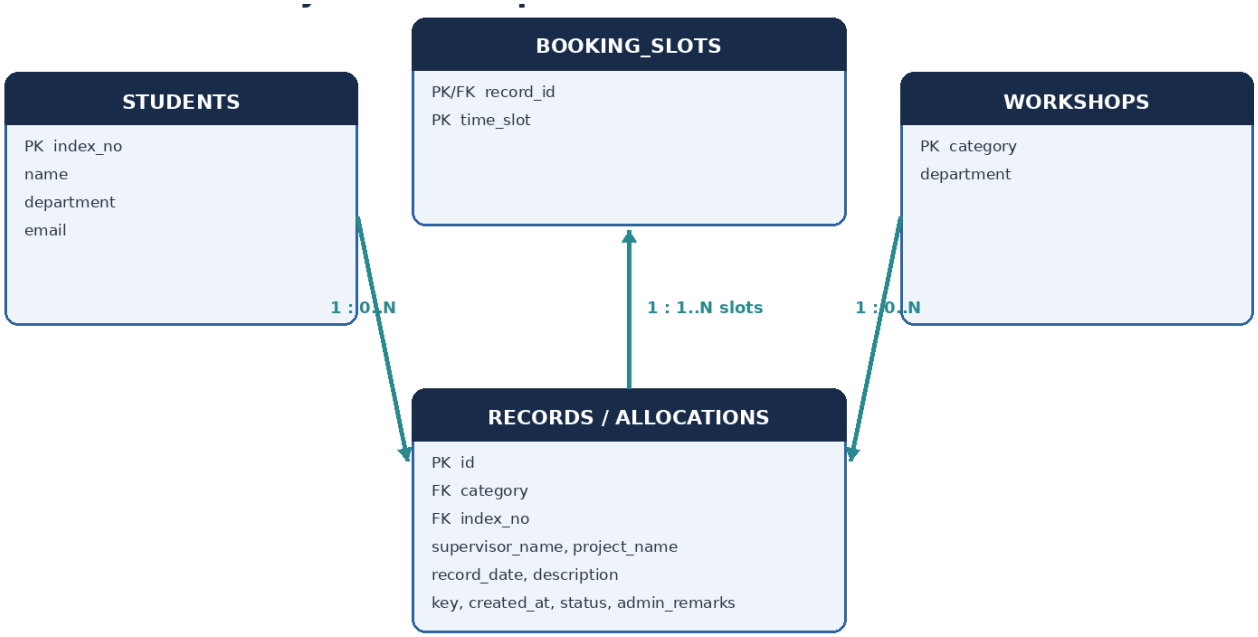


Figure 9 ER diagram for workshops, students, records and booking slots; ensure it matches the submitted SQL

Table	Main columns	Keys / constraints
workshops	category, department	Primary key: category.
students	index_no, name, department, email	Primary key: index_no.
records	id, category, index_no, supervisor_name, project_name, record_date, description, key, created_at, status, admin_remarks	Primary key: id; foreign keys to workshops and students; status constraint.
booking_slots	record_id, time_slot	Composite primary key (record_id, time_slot); foreign key to records with delete cascade; allowed-slot check.

One workshop can have many allocation records, and one student can have many records. One allocation can have one or more booking slots rows. Foreign keys and checks enforce structural rules; application transactions enforce slot availability and key uniqueness. The submitted CREATE TABLE statements and sample data should accompany the final source/database package.

4.6 Class diagram and OOP design

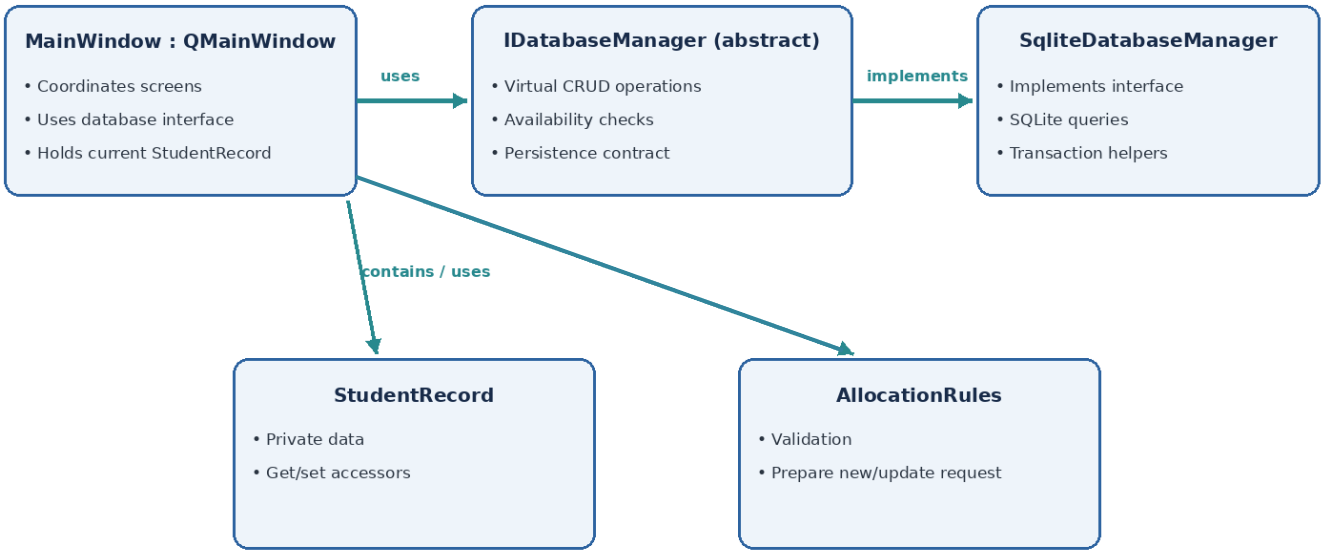


Figure 10 Final class diagram based on the source code; show inheritance and key relationships

Class / component	Responsibility
MainWindow : QMainWindow	Coordinates screens and user actions; uses the database manager and current StudentRecord.
IDatabaseManager	Defines virtual persistence operations, including create, update, delete and availability checks.
SqliteDatabaseManager	Implements persistence using SQLite queries, transactions and related-record operations.
StudentRecord	Stores student/allocation values using private fields and getter/setter accessors.
AllocationRules	Validates fields and prepares new or updated allocation state.
CustomCalendarWidget	Customizes calendar rendering to display allocation-status information.
Utilities	Support file paths, facility metadata and appearance settings.

Encapsulation is demonstrated by private StudentRecord fields and database operations hidden behind the manager interface. Inheritance is used through Qt base classes and the SQLite implementation of IDatabaseManager. Polymorphism allows calls through the database interface to dispatch to the SQLite implementation. Qt signals and slots connect user actions to application behavior.

5. Implementation Details and Key Decisions

Item	Implementation
Language	C++17
GUI framework	Qt 6: Core, GUI, Widgets and SQL
Database	SQLite using the QSQLITE driver
Settings	QSettings for appearance/configuration preferences
Build system	qmake project file (.pro)
Reported build environment	Qt 6.11.2 and MinGW 13.1.0
Package	Windows x64 bundle with required Qt modules and QSQLITE driver
AI-assisted tools	ChatGPT (GPT-6) and Google Gemini (Argon)

Assumptions and Design Considerations

1. Screen Compatibility: The application is designed and optimized for displays with a 16:10 aspect ratio.

2. Database Selection: SQLite was selected based on the assumption of a low data volume, estimated at approximately 10 bookings per month.

3. Security Requirements: The system was designed to handle basic student and booking information, not highly sensitive data. Therefore, advanced security was not a primary development requirement.

4. Passcode Management: For simplicity, certain default passcodes are hardcoded in the source code and can be identified by users with source-code access.

5. Data Privacy: Basic passcode-based access control was considered sufficient for the project's assumed usage environment.

5.1 Allocation and slot rules

- New requests start with Pending status and receive a four-digit Allocation Key.
- The four time slots are fixed in the current implementation.
- Pending and Approved requests reserve their selected slots; Rejected requests release them.
- Availability checks consider the same facility, date and slot; an edited record is excluded from its own conflict check.
- Student updates return the request to Pending; administrator edits preserve the current status according to the supplied implementation notes.
- Allocation Key collisions are handled as errors requiring the operation to be retried with a new key.

5.2 Database consistency

The SQLite implementation uses prepared query bindings and BEGIN IMMEDIATE transactions for writes. Related profile, allocation and slot changes are committed together; failures trigger transaction handling to avoid partially saved changes. Foreign-key checks and delete cascades help maintain relationships between allocation and slot records.

5.3 Email behavior

Pending and rejection actions prepare the relevant email content and open a mailto draft in the computer's default email application. The administrator must review and send the message from that application. SYNCLAB cannot confirm that a message was sent or delivered. The HOD and Dean remain external decision-makers; the software supports status recording and communication rather than independently obtaining their approvals.

5.4 Settings, migration and deployment

The Settings interface supports Light Mode/Dark Mode, background wallpaper changes and facility administrator passcode management. Supplied implementation notes describe safeguards for migrating older data, including backup, staging tables, version checking and foreign-key verification. Before deployment on another computer, writable data paths, bundled Qt libraries and the QSQLITE driver should be checked.

6. Testing Strategy and Results

Testing evidence supplied with the project separates compilation checks, integration behavior, SQL verification and selected UI operations. Test data was isolated from real student records. Reported totals should be checked against the exact test-output files included in the submission package before final hand-in.

Verification activity	Reported result
Production release build	Successful with Qt 6.11.2 / MinGW 13.1.0.
Compiled integration harness	10 of 10 test groups passed; 69 assertions, zero failures.
Independent SQL verification	20 of 20 checks passed.
UI update/delete checks	19 assertions passed.
Sample query outputs	Five SELECT scripts with exported result tables.

6.1 Key test cases

ID	Test	Expected result	Reported
TC-01	Create valid allocation	Saved as Pending; key displayed.	Pass
TC-02	Required field empty	Save refused with validation feedback.	Pass
TC-03	Occupied slot	Conflicting request refused.	Pass
TC-04	Rejected slot release	Slot becomes available again.	Pass
TC-05	Student update	Changes saved; status returns to Pending.	Pass
TC-06	Delete with cascade	Record and child slot rows removed.	Pass
TC-07	Allocation Key collision	Error returned; operation retried.	Pass
TC-08	Wrong credential	Access denied.	Pass

6.2 SQL query verification

- Q1 — Joined allocation details across allocation, student and workshop information.
- Q2 — Workshop usage summary, including categories with no allocations.
- Q3 — Free time slots for a selected date.
- Q4 — Allocation status summary.
- Q5 — Allocation history for one student.

7. Limitations and Possible Future Improvements

Current limitation	Possible improvement
Local SQLite storage; no remote synchronization or web/mobile client.	Introduce server-backed storage if multiple terminals require centralize live data.
Four-digit keys and locally stored passcodes provide lightweight protection; no lockout, password hashing or audit trail.	Add stronger authentication, salted password hashing, rate limiting and audit logs.
Facility catalog and four time periods are fixed in the current source.	Allow authorized configuration of facilities, capacity, closure dates and slot duration.
Email actions open drafts; delivery is not tracked.	Integrate an approved mail service with delivery/error tracking if institutional infrastructure permits.
No routine in-app backup/restore or report export.	Add validated backup/restore and export formats with access controls and retention rules.

Historic bookings do not preserve a separate immutable snapshot of profile details.	Store a booking-time snapshot where historical identity accuracy is required.
Heavy-load performance, accessibility and simultaneous multi-user operation are not established.	Add stress, usability, accessibility and deployment testing before wider use.

7.1 Deployment considerations

Before operational use, the project would need a security review, clear data-retention rules, a tested backup strategy and validation in the intended terminal environment. The current application should be treated as a local prototype rather than a production-grade institutional approval service.

8. Individual Contributions

Member	Index number	Contribution	Demonstration role
Achini	249127	Builds the interface.	UI/UX Designer
Keshan	249003	Designs schema and SQL.	Database Designer
Chamodi	249082	Implements core logic.	OOP Developer
Rashmika	249152	Connects database, UI and logic.	Integrator
Gagamini	249241	Prepares report and tests application.	Documentation & Testing Lead

9. Conclusion

SYNCLAB provides a structured desktop workflow for managing university laboratory and workshop allocations. Students can choose a facility, date and available time slots, enter academic details and create a request that can later be retrieved using an Allocation Key. Facility administrators can inspect records, manage statuses and remarks, filter by date, edit or delete allocations, prepare email drafts.

The design combines Qt Widgets, C++ object-oriented components and a relational SQLite database. Separate allocation and slot records, prepared statements, foreign-key constraints and transaction-protected availability checks support data consistency. The reported build and test results provide evidence for the behaviors exercised in the supplied environment.The application remains a local prototype. Email drafts require external sending, access controls are lightweight, and the application does not independently collect HOD and Dean decisions. Further security, usability, deployment and multi-user testing would be needed before wider operational use.

10. References

- [1] International Organization for Standardization, Information Technology — Programming Languages — C++, ISO/IEC 14882:2017, 2017. [Online]. Available: https://www.iso.org/standard/68564.html
- [2] Qt Group, “Signals & Slots,” Qt 6 Documentation. [Online]. Available: https://doc.qt.io/qt-6/signalsandslots.html
- [3] Qt Group, “Connecting to Databases,” Qt 6 Documentation. [Online]. Available: https://doc.qt.io/qt-6/sql-connecting.html